PROGRAM:

```
#include <stdio.h>
#include <string.h>

char input[100];
int i = 0;

void E();
void T();
void F();

void E()
{
   T();

   if (input[i] == '+')
   {
      i++;
      E();
   }
}

void T()
{
   F();

   if (input[i] == '*')
   {
      i++;
      T();
   }
}

void F()
{
   if (input[i] >= 'a' && input[i] <= 'z')
   {
      i++;
   }
}

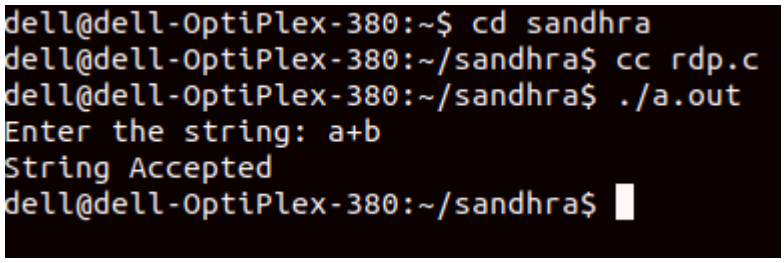
int main()
{
   printf("Enter the string: ");
   scanf("%s", input);

   E();

   if (input[i] == '\0')
      printf("String Accepted\n");
   else
      printf("String Rejected\n");

   return 0;
}
```

OUTPUT:


dell@dell-OptiPlex-380:~$ cd sandhra
dell@dell-OptiPlex-380:~/sandhra$ cc rdp.c
dell@dell-OptiPlex-380:~/sandhra$ ./a.out
Enter the string: a+b
String Accepted
dell@dell-OptiPlex-380:~/sandhra$

PROGRAM:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char stack[MAX][20];
char input[MAX];
char lhs[MAX][10];
char rhs[MAX][50];

int top = -1;
int n;

/* Display stack */
void printStack()
{
   int i;

   for (i = 0; i <= top; i++)
      printf("%s", stack[i]);
}

/* Check and perform reduction */
int reduce()
{
   int i, j, len;
   char temp[100];

   for (i = 0; i < n; i++)
   {
      len = strlen(rhs[i]);

      /* Build string from top stack symbols */
      temp[0] = '\0';

      for (j = top; j >= 0; j--)
      {
         char current[100] = "";

         strcat(temp, stack[j]);

         if (strcmp(temp, rhs[i]) == 0)
         {
            /*
             * Count how many stack symbols
             * were used for the RHS.
             */
            int count = top - j + 1;

            top = j;

            strcpy(stack[top], lhs[i]);
```

```
            printf("\n");
            printStack();
            printf("\t\tReduce %s->%s", lhs[i], rhs[i]);

            return 1;
         }

         /*
          * If the current string is longer than
          * the RHS, stop checking this production.
          */
         if (strlen(temp) >= len)
            break;
      }
   }

   return 0;
}

/* Get next token from input */
int getToken(char token[])
{
   static int pos = 0;

   if (input[pos] == '\0')
      return 0;

   /* Recognize "id" as one token */
   if (input[pos] == 'i' && input[pos + 1] == 'd')
   {
      strcpy(token, "id");
      pos += 2;
   }
   else
   {
      token[0] = input[pos];
      token[1] = '\0';
      pos++;
   }

   return 1;
}

int main()
{
   int i;
   char production[MAX];
   char token[20];

   printf("Enter number of productions: ");
   scanf("%d", &n);

   printf("\nEnter productions in the form A->B\n");
   printf("Example: E->E+E\n\n");
```

```
for (i = 0; i < n; i++)
{
   printf("Production %d: ", i + 1);
   scanf("%s", production);

   char *arrow = strstr(production, "->");

   if (arrow != NULL)
   {
      int lhsLength = arrow - production;

      strncpy(lhs[i], production, lhsLength);
      lhs[i][lhsLength] = '\0';

      strcpy(rhs[i], arrow + 2);
   }
   else
   {
      printf("Invalid production format!\n");
      i--;
   }
}

printf("\nEnter input string: ");
scanf("%s", input);

printf("\n\nSHIFT REDUCE PARSER");
printf("\n====================\n");

printf("\nStack\t\tAction");

/*
 * Shift tokens one by one.
 */
while (getToken(token))
{
   /* SHIFT */
   strcpy(stack[++top], token);

   printf("\n");
   printStack();
   printf("\t\tShift %s", token);

   /* REDUCE */
   while (reduce());
}

/* Final reductions */
while (reduce());

/*
 * Accept if stack contains start symbol.
 * First production's LHS is treated as start symbol.
 */
if (top == 0 && strcmp(stack[0], lhs[0]) == 0)
{
```

```
        printf("\n\nString Accepted!\n");
    }
    else
    {
        printf("\n\nString Rejected!\n");
    }

    return 0;
}
```

OUTPUT:

